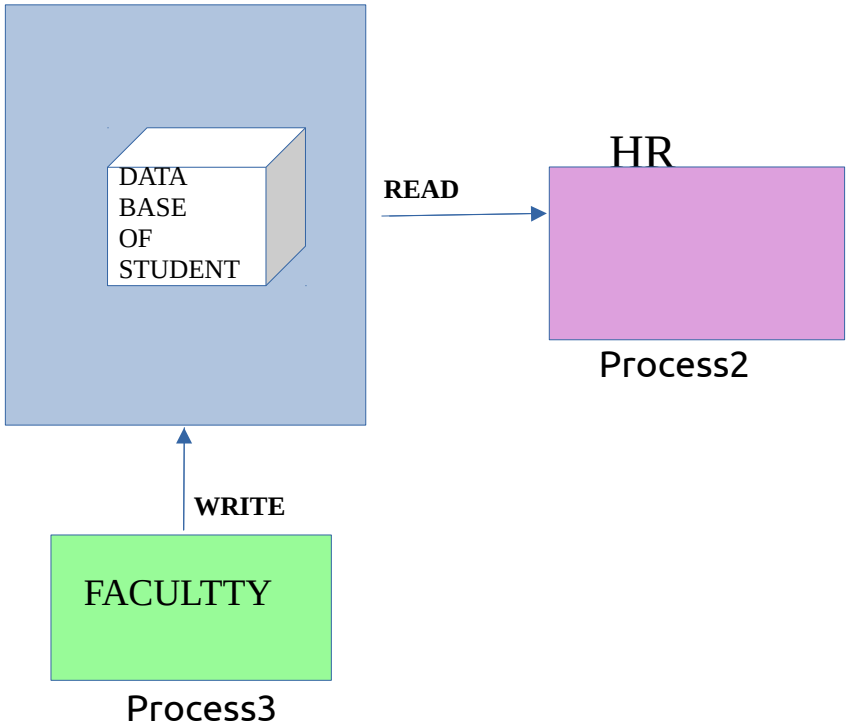
RECORD LOCK

The Operating system allow multiple processes to read and write the same file concurrently. It is a means of data sharing among processes i.e, common resource area.

What is record lock?
FILE/File region beign locked by a process so that others can't acess that region concurrently.

A Scenario:
In Vtrack server Student is checking data, and HR is fetching student list for campus drive filter top N students. At the same time faculty trying to update the marks of the same list of students.

Is the data used by HR in this situation accurate??



Not only this we can give examples like ticket booking, excel sheet, using same file by the two process and so on.

It is a concept of job synchronisation/data safety mechanism during common resource access.

Why the need to lock records?
It is needed in some applications like database access where no other process can write or read a record while a process is accessing a recod. To provide this capability for processes that need it, record locking.

What is record locking?
Record locking is used to describe the ability of a process to prevent other processes from modifying a region of a record while the first process is reading or modifying that portion of the record.

These locks can be imposed on the whole record or a portion of it.

We mentioned two types of locks:

1.A shared read lock (l_type of F_RDLCK)

2.An exclusive write lock (F_WRLCK).

The basic rule is that any number of processes can have a shared read lock on a given byte, but only one process can have an exclusive write lock on a given byte.

Through the fcntl() we can do record locking.

The function allowed callers to lock byte ranges in a file, from the entire file down to a single byte within the file.

```
#include <fcntl.h>
int fcntl(int filedes, int cmd, ... /* struct  flock *flockptr */ );
```

For record locking, cmd is F_GETLK, F_SETLK, or F_SETLKW. The third argument (which we'll call flockptr) is a pointer to an flock structure.

```
   struct flock {

 short l_type;   /* F_RDLCK, F_WRLCK, or F_UNLCK */

  short l_whence; /* SEEK_SET, SEEK_CUR, or SEEK_END */

     off_t l_start;  /* offset in bytes, relative to l_whence */

     off_t l_len;    /* length, in bytes; 0 means lock to EOF */

      pid_t l_pid;    /* returned with F_GETLK */

 };
```

This structure describes

- The type of lock desired: F_RDLCK (a shared read lock), F_WRLCK (an exclusive write lock), or F_UNLCK (unlocking a region)
- The starting byte offset of the region being locked or unlocked (l_start and l_whence)
- The size of the region in bytes (l_len)
- The ID (l_pid) of the process holding the lock that can block the current process (returned by F_GETLK only)

There are numerous rules about the specification of the region to be locked or unlocked.

- the l_whence member is specified as SEEK_SET, SEEK_CUR, or SEEK_END.
-

We can now describe the three commands for the fcntl function.

F_SETLK F_SETLKW F_GETLK Use Sets a file lock. Do not block if this cannot succeed immediately. Sets a file lock and blocks the calling process until the lock is acquired. Queries as to which process locked a specified region of a file.